

AtariX - Änderungen von Version 0.3 bis Version 0.6.5

Stand: 30. August 2026

Diese Übersicht wurde durch den Vergleich des ursprünglichen Quellstands 0.3 mit den archivierten Quellständen 0.4, 0.4B, 0.4C, 0.4D, 0.4E und 0.4F sowie den veröffentlichten Quellständen 0.5, 0.6, 0.6.4 und 0.6.5 erstellt.

VERSION 0.4 - GRUNDLEGENDE MODERNISIERUNG UND APPLE-SILICON- PORTIERUNG

Plattform und Buildsystem

- AtariX wurde von einer historischen 32-Bit-Anwendung auf moderne 64-Bit-Systeme portiert.
- Native Unterstützung für Apple Silicon (arm64) wurde ergänzt.
- Die Intel-64-Bit-Architektur x86_64 bleibt im Buildsystem enthalten.
- Die nicht mehr unterstützte 32-Bit-Architektur i386 wurde aus der aktuellen Buildkonfiguration entfernt.
- Das minimale macOS-Ziel wurde auf macOS 11.0 angehoben.
- Der C++-Sprachstandard wurde auf C++17 und die Standardbibliothek auf libc++ umgestellt.
- Die Xcode-Projektversion wurde von 0.3 auf 0.4 geändert.
- Eine GitHub-Actions-Konfiguration für einen arm64-Kompilier- und Linktest wurde ergänzt.
- Buildausgaben im Verzeichnis "build" wurden in .gitignore aufgenommen.

SDL 2

- Das veraltete, nur für Intel geeignete SDL2.framework wurde aus dem Quellpaket entfernt.
- Das Skript scripts/bootstrap-sdl2.sh wurde ergänzt. Es lädt das offizielle universelle SDL 2.32.10 Framework mit arm64- und x86_64-Code herunter und installiert es im Projekt.
- Das Installationsverfahren erhält die symbolischen Verknüpfungen des Frameworks auch unter aktuellen macOS-Versionen. Dadurch wird der zuvor aufgetretene "ditto: ... Is a directory"-Fehler vermieden.
- Die Verwendung der nicht öffentlichen und in modernen SDL- Versionen entfernten Funktion SDL_KeyboardActivate() wurde beseitigt.
- Tastaturereignisse werden nun über die öffentlichen SDL-Funktionen SDL_EventState() und SDL_FlushEvent() aktiviert, deaktiviert und bereinigt.

Threading und Synchronisation

- Die veralteten Carbon Multiprocessing Services wurden entfernt.
- MPTask, MPEvent, MPQueue und MPCriticalRegion wurden durch moderne C++17-Mechanismen ersetzt.
- Die Emulation verwendet nun std::thread.
- Ereignisse und Wartezustände werden mit std::condition_variable umgesetzt.
- Kritische Bereiche verwenden std::recursive_mutex.
- Hierfür wurde die neue Hilfsdatei ModernSync.h eingeführt.
- Start, Stopp und Beendigung des Emulationsthreads wurden an die neue Threadverwaltung angepasst.

64-Bit- und ARM64-Kompatibilität

- Verbliebene Annahmen über 32-Bit-Hostzeiger wurden entfernt.
- 68k-Parameter und Rückrufparameter werden als Offsets innerhalb des emulierten Atari-Speichers interpretiert und über die RAM- Basisadresse in gültige Hostzeiger umgesetzt.
- Nicht mehr benötigte PowerPC-Hostadressfelder werden nicht länger mit auf 32 Bit gekürzten Zeigern befüllt.
- Speicherbereiche und Offsets werden vor dem Zugriff auf ihre gültigen Grenzen geprüft.
- Nicht portable Compiler-Pragmas für gepackte Strukturen wurden durch portable "#pragma pack(push, 1)"- und "#pragma pack(pop)"- Anweisungen ersetzt. Dadurch bleibt das erwartete Atari-/68k-Speicherlayout auf ARM64 erhalten.
- Die Abhängigkeit vom alten Carbon-Datentyp Point wurde entfernt.
- Für Atari-Mauskoordinaten wird nun ein eigener 16-Bit-Datentyp AtariPoint verwendet.
- Die Erkennung der Hostarchitektur wurde um Apples __arm64__ - Definition ergänzt. Im Informationsdialog wird die Architektur als "arm64" ausgewiesen.

Fenster, Darstellung und Vollbildmodus

- Das AtariX-Fenster kann nun in der Größe verändert werden.
- Unterstützung für HiDPI-Fenster wurde aktiviert.
- Ein Vollbildmodus wurde ergänzt.
- Der Vollbildmodus kann über das Ansichtsmenü, Control-Command-F oder Alt-Return ein- und ausgeschaltet werden.
- Escape beendet den von AtariX aktivierten Vollbildmodus.
- Der Menüeintrag wurde in die englische, deutsche und französische Benutzeroberfläche aufgenommen: Englisch: Toggle Full Screen Deutsch: Vollbildmodus Französisch: Plein écran
- Die Darstellung verwendet die logische SDL-Rendergröße.
- Beim Vergrößern des Fensters und im Vollbild bleibt das ursprüngliche Atari-Seitenverhältnis erhalten.
- Nicht verwendete Bildschirmbereiche werden schwarz dargestellt.
- Fensterereignisse wie Größenänderung, Wiederherstellung und Maximierung aktualisieren die Darstellung korrekt.
- Mauskoordinaten des macOS-Fensters werden über SDL_RenderWindowToLogical() in die Koordinaten des emulierten Atari-Bildschirms zurückgerechnet.

Mauszeiger

- Im Vollbildmodus wird der macOS-Mauszeiger verborgen, sodass nur der Atari-Mauszeiger sichtbar bleibt.
- Beim Verlassen des Vollbildmodus, beim Fokusverlust und beim Beenden von AtariX wird der macOS-Mauszeiger wieder eingeblendet.
- Die Zeigersichtbarkeit wird bei relevanten Fensterereignissen aktualisiert, damit der Hostzeiger nicht unbeabsichtigt verborgen bleibt.

Gemappte Laufwerke und MacXFS

- Die alten, erzwungenen M_DRV_READONLY-Zuweisungen wurden entfernt.
- Als Atari-Laufwerke eingebundene macOS-Ordner können damit beschrieben werden, sofern die macOS-Dateirechte dies erlauben.
- XFSDDevFunctions() prüft den Offset eines emulierten Dateideskriptors, bevor daraus ein Hostzeiger gebildet wird.

- Der native Laufwerkszeiger wird bei Geräteoperationen aus den Informationen des emulierten Dateideskriptors rekonstruiert.
- Damit wurde der ARM64-Absturz EXC_BAD_ACCESS an Adresse 0x8 behoben, der beim Start mit beschreibbaren Laufwerken auftreten konnte.

Versions- und Informationsanzeige

- Die Versionsinformation wird direkt und sicher aus dem App-Bundle gelesen.
- Eine zuvor mögliche fehlerhafte Ausgabe "(null)" wurde beseitigt.
- Der Informationsdialog nennt die Aktualisierung für ARM64/SDL sowie den Bearbeiter oberhalb des Kompilierungsdatums.

Dokumentation und Paketierung

- APPLE_SILICON.md mit Voraussetzungen, Buildanleitung, Prüfkommandos und Beschreibung der Portierungsarbeiten wurde ergänzt.
- PACKAGE-NOTE.md mit den wichtigsten Hinweisen zum Quellpaket wurde ergänzt.
- README.md wurde auf die Unterstützung aktueller 64-Bit-Intel- und Apple-Silicon-Macs angepasst.
- Das Quellpaket enthält bewusst keine vorgefertigte AtariX.app und kein fest eingebettetes SDL2.framework. Das Framework wird reproduzierbar durch das Bootstrap-Skript bereitgestellt.

VERSION 0.4B - KORREKTE VERSIONSKENNZEICHNUNG

- Die Versionsnummer im Xcode-Projekt und in den Paketinformationen wurde auf 0.4B geändert.
- Gegenüber 0.4 enthält dieser Zwischenstand keine zusätzliche funktionale Quellcodeänderung.

VERSION 0.4C - NATIVE macOS-VOLLBILD- UND MAUSZEIGERKORREKTUREN

- Der native macOS-Vollbildmodus, der über den grünen Fensterknopf einen eigenen Vollbild-Space öffnet, wird nun ebenfalls erkannt.
- Weil SDL diesen Zustand nicht immer mit SDL_WINDOW_FULLSCREEN_DESKTOP kennzeichnet, erkennt AtariX zusätzlich, ob das Fenster mindestens etwa 90 Prozent des Bildschirms ausfüllt.
- Der macOS-Mauszeiger wird auch im nativen macOS-Vollbild-Space zuverlässig verborgen.
- Nach einem Wechsel des macOS-Spaces wird die Zeigersichtbarkeit bei Mausbewegungen erneut gesetzt.
- Die zuvor ignorierte Einstellung "Host-Mauszeiger verbergen" wird wieder übernommen und angewendet.
- Die Xcode-Projektversion wurde auf 0.4C geändert.

VERSION 0.4D - ZUVERLÄSSIGE SCHREIBVORGÄNGE AUF GEMAPPTEN LAUFWERKEN

- Für jede von MacXFS geöffnete Hostdatei wird die Zuordnung zwischen nativem macOS-Dateideskriptor und Atari-Laufwerk in einer Host-Datenstruktur gespeichert.
- Diese Information liegt damit nicht mehr in einem auf 32 Bit begrenzten emulierten 68k-Datenfeld.
- Alle nachfolgenden Geräteoperationen können den korrekten Laufwerkskontext über den nativen Dateideskriptor wiederherstellen.
- Beim Schließen einer Datei wird die gespeicherte Zuordnung entfernt.
- Für ältere oder geerbte Deskriptoren bleibt die Rekonstruktion aus dem emulierten fd_dmd-Feld als Kompatibilitätslösung erhalten.

- Damit wurde die Regression aus 0.4C behoben, bei der Verzeichnisse angelegt werden konnten, kopierte Dateien aber wegen TOS_EDRIVE bei der Schreiboperation wieder verworfen wurden.
- Die Xcode-Projektversion wurde auf 0.4D geändert.

VERSION 0.4E - KOMPATIBLE DATEIRECHTE UND DATEIEIGENTÜMER

- `xfs_fchmod()` liefert nach einem erfolgreich ausgeführten `chmod()` nun `TOS_E_OK` statt fälschlich `TOS_EINVFN`.
- Ungültige Dateinamen und echte Hostfehler werden weiterhin als passende Atari-/MiNT-Fehler zurückgegeben.
- `xfs_fchown()` prüft, ob die betreffende Datei existiert.
- Atari-Benutzer- und Gruppen-IDs werden nicht auf normale macOS- Dateien übertragen. Dadurch werden Root-Rechte, falsche Eigentümer und ein möglicher Verlust des Dateizugriffs vermieden.
- Bei existierender Datei wird `Fchown` als kompatibler No-op erfolgreich bestätigt, wie bei Atari-Dateisystemen ohne Eigentümerverwaltung.
- Jinnee wertet `Fchmod` und `Fchown` dadurch nicht mehr irrtümlich als fehlgeschlagenen Kopiervorgang.
- Die Xcode-Projektversion wurde auf 0.4E geändert.

VERSION 0.4F - ABSCHLUSS DER DATEIKOPIERKORREKTUREN

- Die veralteten Finder-Metadaten "Type" und "Creator" werden auf modernen macOS-Dateisystemen kompatibel behandelt.
- `FMACGETTYCR` liefert acht Nullbytes als leere Type-/Creator- Information und bestätigt die Abfrage mit `TOS_E_OK`.
- `FMACSETTYCR` wird als optionale, heute nicht mehr erforderliche Metadatenoperation akzeptiert und als erfolgreicher No-op bestätigt.
- Diese Behandlung wurde sowohl für pfadbezogene als auch für dateideskriptorbezogene `ioctl`-Aufrufe umgesetzt.
- `xfs_sync()` bestätigt Synchronisationsanforderungen für gemappte macOS-Verzeichnisse nun mit `TOS_E_OK`. Die eigentlichen Schreibvorgänge werden bereits unmittelbar an macOS übergeben und Dateien normal geschlossen.
- Zusätzliche Debugausgaben für `dev_ioctl()` zeigen Dateideskriptor, Kommandonummer und Pufferadresse und erleichtern die Fehlersuche.
- Damit wurde der letzte Kopierfehler behoben: Jinnee löschte zuvor eine bereits vollständig geschriebene Zieldatei wieder, weil die optionale Finder-Metadatenoperation den Fehler -25 zurückgab.
- Das Anlegen und Kopieren von Dateien zwischen Verzeichnissen des gemappten Laufwerks C: wurde anschließend erfolgreich praktisch überprüft.
- Die Xcode-Projektversion wurde auf 0.4F geändert.

VERSION 0.5 - AKTUALISIERTER MUSASHI-KERN UND ARM64-KORREKTUREN

Musashi-680x0-Emulator

- Der zuvor enthaltene ältere Musashi-Kern wurde auf den Upstream- Stand mit dem Commit 313ebf1bd9f4d0d93341eb5ce21fd8a119e9dbdd vom 8. März 2026 aktualisiert.
- Die neu erzeugten Opcode-Dateien weisen die Musashi- Generatorversion 4.60 aus.
- Die früher getrennten Opcode-Quelldateien `m68kopac.c`, `m68kopdm.c` und `m68kopnz.c` wurden durch die aktuelle, gemeinsam erzeugte `m68kops.c` ersetzt.

- Die für den neuen Kern erforderlichen Dateien m68kcpu.c, m68kmmu.h sowie die SoftFloat-Quellen wurden in das Xcode-Projekt aufgenommen.
- AtariX emuliert weiterhin einen MC68020. Die nicht benötigte Emulation von 68030, 68040 und PMMU bleibt deaktiviert.
- M68K_USE_64_BIT bleibt aus Kompatibilitätsgründen abgeschaltet.

AtariX- und MagicMac-Kompatibilität

- Die von MagicMac verwendeten AtariX-Hostcall-Opcodes 0x00c0 und 0x00c1 wurden in die aktuelle Musashi-Opcode-Tabelle übernommen.
- Die Hostcall-Funktionen erhalten weiterhin Zugriff auf die Basis des emulierten Atari-Speichers.
- Die Anbindung wurde auf die aktuellen Musashi-Schnittstellen m68k_execute(), m68k_end_timeslice(), m68k_set_irq() und m68k_pulse_bus_error() umgestellt.
- Für natfeat.h wurde eine kompatible INLINE-Definition ergänzt, weil der aktuelle Musashi-Kern das frühere Makro nicht mehr bereitstellt.

Thread-Unterbrechung und Busfehler

- Eine eigene threadübergreifende Stoppanforderung wird an jeder Instruktionsgrenze geprüft. Damit bleibt das bisherige Verhalten bei Maus-, Timer- und VBL-Ereignissen erhalten.
- Busfehler werden unter Musashi nun synchron ausgelöst, solange der von m68k_execute() eingerichtete Fehlerkontext noch gültig ist.
- Zuvor konnte ein verzögert ausgelöster Busfehler auf ARM64 in einen bereits verlassenen Stack-Kontext zurückspringen und AtariX beim Start der Emulation zum Absturz bringen.
- Durch die synchrone Behandlung startet MagiC mit dem aktualisierten Musashi-Kern stabil.

Framebuffer und Farbdarstellung

- Der 32-Bit-macOS-Framebuffer speichert Pixel in nativer Little-Endian-Reihenfolge, während der 68k-Gast Big Endian verwendet.
- Bei Host-Endian-Videospeicher werden deshalb 8-Bit-Zugriffe nun mit Adress-XOR 3 und 16-Bit-Zugriffe mit Adress-XOR 2 abgebildet.
- 32-Bit-Zugriffe bleiben unverändert.
- Damit wurde der Farbfehler beseitigt, bei dem Fenster während des Verschiebens fortlaufend ihre Farben wechselten und anschließend die zuletzt angezeigte Fehlfarbe behielten.

Version, Dokumentation, Build und Veröffentlichung

- Die interne Versionskennung und die Xcode-Projektversion wurden auf 0.5 geändert.
- MUSASHI.md dokumentiert den festgelegten Upstream-Commit, die AtariX-spezifischen Erweiterungen und die Regenerierung der Opcode-Dateien.
- Der GitHub-Actions-Build erzeugt nun automatisch das Paket AtariX-0.5-Apple-Silicon.zip.
- Der abschließende Build wurde auf macOS 15 erfolgreich als native arm64-Anwendung kompiliert. Das eingebettete SDL2-Framework enthält weiterhin arm64- und x86_64-Code.
- Der praktische Test bestätigte den Start von AtariX und MagiC sowie eine stabile Farbdarstellung beim Verschieben von Fenstern.
- Version 0.5 wurde als dauerhaftes GitHub Release v0.5 mit der kompilierten Apple-Silicon-Anwendung veröffentlicht.

VERSION 0.6 - KORREKTE 32-BIT-FARBEN UND VERBESSERTE macOS-VERTEILUNG

32-Bit-True-Color-Darstellung

- Die SDL-Pixelmasken für den direkten 32-Bit-Bildschirm und die Konvertierungsoberfläche wurden auf das unter Little Endian erwartete RGB-Layout 0xAARRGGBB korrigiert.
- Die vorherige BGR-Zuordnung vertauschte Rot und Blau. Dadurch erschienen beispielsweise rote oder magentafarbene Bildanteile blau beziehungsweise cyan.
- PhotoLine- und Calamus-Bildinhalte werden nun mit der korrekten Farbkanalzuordnung dargestellt.

VDI-Farbtabelle und Speicherzugriffe

- AtariVsetRGB() erhöht den Zeiger auf die Farbtabelle nicht mehr doppelt. Zuvor wurde jeder zweite Paletteneintrag übersprungen und bei größeren Anfragen konnte über das Ende der 256 Einträge hinaus geschrieben werden.
- AtariVgetRGB() erhöht den Atari-Ausgabezeiger nun um vier Byte pro 00rrggb-Farbwert. Dadurch überlappen aufeinanderfolgende Farbwerte nicht mehr.
- Ungültige Startindizes, leere Anfragen und über das Tabellenende hinausreichende Bereiche werden vor dem Zeigerzugriff abgefangen.
- Die Umwandlung zwischen VDI-Farbwerten und dem Hostformat verwendet konsistent die Kanalreihenfolge 0xAARRGGBB.

Versionskennung, Signierung und Veröffentlichung

- Die interne Versionskennung und die Xcode-Projektversion wurden auf 0.6 geändert.
- Der GitHub-Build prüft neben der arm64-Architektur des Programms und des eingebetteten SDL2-Frameworks auch die tatsächlich erzeugte App-Versionsnummer 0.6.
- Das eingebettete SDL2-Framework wird nach dem Kopieren erneut ad-hoc signiert. Anschließend wird das vollständige AtariX-App-Bundle signiert und mit codesign --verify --deep --strict geprüft.
- Damit wird die frühere macOS-Meldung beseitigt, die heruntergeladene App sei beschädigt. Ohne Apple-Developer-ID und Notarisierung bleibt lediglich die übliche einmalige Sicherheitsbestätigung für extern geladene Anwendungen möglich.
- GitHub Actions erzeugt das Paket ATARIX-A64-0.6-Release.zip und veröffentlicht es dauerhaft mit dem Tag ATARIX-A64-0.6-Release.
- Der finale macOS-15-Build, die Architekturprüfung, die Versionsprüfung, die Signaturprüfung und die Paketierung wurden erfolgreich abgeschlossen.

Praktische Prüfung

- MagiC und Jinnee starten stabil; Fenster lassen sich ohne wechselnde oder dauerhaft verfälschte Flächen bewegen.
- Die korrekte True-Color-Wiedergabe wurde anhand realer Bilder in PhotoLine und Calamus überprüft.

VERSION 0.6.4 - KORRIGIERTE VOLLBILDMAUS UND OPTIMIERTE EREIGNISVERARBEITUNG

Vollbild-Maussteuerung unter macOS

- Der Fehler wurde behoben, bei dem sich der Atari-Mauszeiger nach dem Wechsel in den Vollbildmodus nur innerhalb der vorherigen Fensterfläche bewegen ließ.
- Im Vollbild verarbeitet AtariX relative SDL-Mausbewegungen und führt daraus eine eigene virtuelle Atari-Mausposition über die gesamte emulierte Bildschirmfläche.
- Der Fenstermodus verwendet weiterhin absolute Fensterkoordinaten. Damit bleibt das bisherige Verhalten außerhalb des Vollbildmodus erhalten.

- Die Mausbegrenzung wird vor dem Vollbildwechsel freigegeben und erst nach der tatsächlichen Größenänderung für den relativen Modus neu aufgebaut. Dies umgeht eine bekannte SDL-/macOS-Begrenzung auf die alte Fenstergröße.
- Sowohl der native macOS-Vollbildmodus über den grünen Fensterknopf als auch Control-Command-F werden unterstützt.

Flüssigkeit und Skalierung

- Relative Mousedeltas werden nicht mehr doppelt durch SDL und AtariX skaliert.
- AtariX übernimmt die Skalierung einmalig mit Gleitkomma-Akkumulation. Bruchteile kleiner Bewegungen bleiben dadurch erhalten und der Zeiger bewegt sich gleichmäßiger.
- Die macOS-Systembeschleunigung wird auch im relativen Vollbildmodus verwendet, sodass Geschwindigkeit und Bewegungsgefühl dem Fenstermodus näher entsprechen.

Vollbildererkennung und Emulatorunterbrechungen

- Ein großes oder maximiertes Fenster wird nicht mehr allein aufgrund einer 90-Prozent-Schwelle als Vollbild behandelt.
- Der native Vollbildzustand wird anhand der nahezu exakten Übereinstimmung von Fensterposition, Fenstergröße und Displaygrenzen erkannt.
- Synthetische oder unveränderte Mauspositionen lösen keinen unnötigen Abbruch eines Musashi-Zeitschritts und keinen zusätzlichen MagiC-Mausinterrupt mehr aus.

Renderer und praktische Prüfung

- Nach Größen-, Wiederherstellungs- und Vollbildänderungen werden SDL-Viewport und logische Koordinatentransformation neu auf die aktuelle Fenstergeometrie abgestimmt.
- Der Vollbildzugriff auf die gesamte emulierte Bildschirmfläche und die flüssigere Mausbewegung wurden praktisch unter MagiC geprüft.

Version, Build und Veröffentlichung

- Die interne Versionskennung und die Xcode-Projektversion wurden auf 0.6.4 geändert.
- Der GitHub-Actions-Build prüft die arm64-Architektur, die App-Version 0.6.4 und die Signatur des vollständigen App-Bundles.
- Das reguläre Paket wird als ATARIX-A64-0.6.4-Release.zip mit dem gleichnamigen GitHub-Tag veröffentlicht.

VERSION 0.6.5 - SAUBERES AUSSCHALTEN VON MagiC UND ATARIX

Gastgesteuertes Ausschalten

- Der Fehler wurde behoben, bei dem AtariX nach der Auswahl von Ausschalten im Atari-Desktop nicht beendet wurde, sondern mit einem eingefrorenen Emulationsfenster geöffnet blieb.
- MagiC erreichte den AtariX-Hostcall AtariExit() bereits korrekt. Dieser beendete zuvor jedoch nur den 68k-Emulationsthread; die SDL-Ereignisschleife und die macOS-Anwendung erhielten kein Beenden-Signal.
- AtariExit() leitet den Gast-Shutdown nun thread-sicher als SDL-User-Event an die Host-Ereignisschleife weiter.
- Die SDL-Ereignispumpe wird kontrolliert verlassen. Anschließend fordert AtariX auf dem Cocoa-Hauptthread das reguläre Beenden der macOS-Anwendung an.
- Ein unmittelbarer Prozessabbruch mit exit() wird vermieden. Dadurch bleibt der normale Beendigungspfad der Anwendung erhalten.

Abgrenzung und praktische Prüfung

- Die Desktop-Funktion Ende wurde nicht verändert. Sie beendet weiterhin nur den gestarteten Desktop; ein anschließender Desktop-Neustart wird durch die MagiC-beziehungsweise MAGX.INF-Konfiguration bestimmt.
- Das vollständige Beenden von AtariX über Ausschalten wurde mit MagiC praktisch geprüft und bestätigt.

Version, Build und Veröffentlichung

- Die interne Versionskennung und die Xcode-Projektversion wurden auf 0.6.5 geändert.
- Der GitHub-Actions-Build prüft die arm64-Architektur, die App-Version 0.6.5 sowie die Ad-hoc-Signatur des vollständigen App-Bundles.
- Das reguläre Paket wird als ATARIX-A64-0.6.5-Release.zip mit dem gleichnamigen GitHub-Tag veröffentlicht.

ERGEBNIS GEGENÜBER VERSION 0.3 BIS VERSION 0.6.5

- nativ auf Apple-Silicon-Macs als arm64-Anwendung kompilierbar,
- weiterhin für x86_64 konfiguriert,
- auf moderne C++17-Thread- und Synchronisationsmechanismen umgestellt,
- von wesentlichen 32-Bit- und Carbon-Abhängigkeiten befreit,
- mit einem aktuellen universellen SDL-2-Framework baubar,
- in einem veränderbaren Fenster sowie mit vollständig nutzbarer und flüssiger Maussteuerung im macOS-Vollbild verwendbar,
- mit beschreibbaren gemappten macOS-Verzeichnissen und vollständigen Jinnee-Dateioperationen verwendbar,
- mit einem aktuellen, reproduzierbar festgelegten Musashi-Kern und den MagicMac-Hostcall-Opcodes ausgestattet,
- gegen den ARM64-Absturz bei verzögerten Busfehlern abgesichert,
- bei 8-, 16- und 32-Bit-Zugriffen auf den macOS-Framebuffer korrekt abgebildet,
- bei VDI-Farbtabelle gegen überlappende, übersprungene und außerhalb der Tabelle liegende Zugriffe abgesichert,
- mit korrekter RGB-Kanalzuordnung für 32-Bit-True-Color-Bilder ausgestattet und
- über den MagiC-Befehl Ausschalten vollständig und kontrolliert beendbar sowie
- als geprüftes, ad-hoc signiertes Apple-Silicon-Paket dauerhaft auf GitHub veröffentlicht.

WEITERHIN GELTENDE HINWEISE

- Der Ordner für das Atari-Laufwerk C: (beispielsweise MAGIC_C) muss weiterhin angelegt und in AtariX als Laufwerk ausgewählt werden.
- Das SDL2.framework muss vor einem lokalen ersten Build einmal mit scripts/bootstrap-sdl2.sh installiert werden.
- Ein vollständiger lokaler Build und Funktionstest muss auf macOS erfolgen, da AtariX von AppKit, Core Foundation und dem macOS-SDL-Framework abhängt.
- AtariX 0.6.5 emuliert weiterhin einen MC68020 ohne MMU. Die 68030-, 68040- und PMMU-Emulation ist nicht aktiviert.
- Das veröffentlichte 0.6.5-Paket ist ad-hoc signiert, jedoch nicht mit einer Apple-Developer-ID notarisiert. macOS kann deshalb beim ersten Start einmalig eine Bestätigung verlangen. In diesem Fall im Finder mit der rechten Maustaste auf AtariX.app klicken, Öffnen wählen und bestätigen.